

privacy the government will ask about

The PS asks for this in its own words. This is the section where a weak answer costs the most, because scalability is the phrase the problem statement chose.

Q6.1 — "Your PS demands a scalable backend. Describe what you have today, honestly, and what breaks first."

Think. Give the breakage ladder in order. A team that can name what breaks *first* has load- tested its own thinking; a team that says "we'll use Kubernetes" has not.

Today. One FastAPI service (`web/api`) + one AI service (`ai/`), Postgres, S3-compatible object store. `web/` calls `ai/` over HTTP and **never imports across that line** — separate deploy units, which is the property that lets the AI service scale on GPUs while the API scales on CPUs.

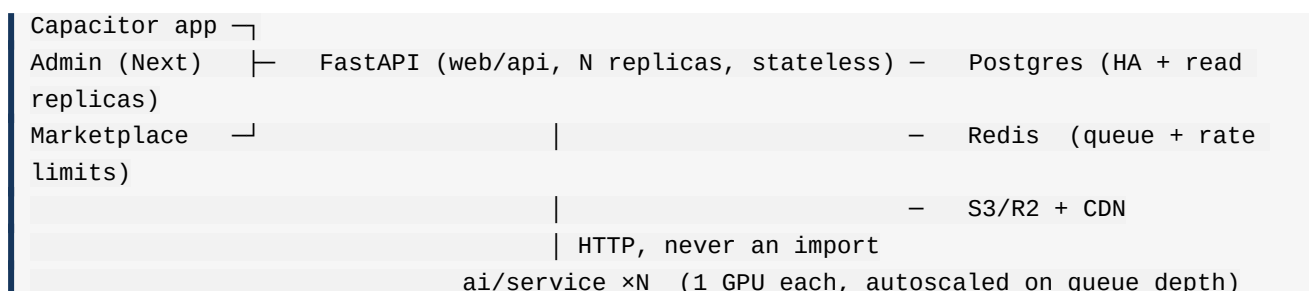
What breaks first, in order, with the number:

#	Limit	Threshold	Fix	Status
1	Object-store round trips	~7 s per photo from Sydney	Move to <code>ap-south-1</code> ; <code>asyncio.gather</code> the 7 puts; <code>ai/</code> writes storage directly	Already hit. S3 switched off for demo 2026-09-02
2	DB latency	200–406 ms per query × several per request from Sydney	Same region move	Same fix, same setting
3	DB connection pool	Each request holds a connection ~1–2 s at that latency ⇒ a pool of 10 saturates near 5–10 rps	Region move collapses hold time; then PgBouncer	Not tuned
4	Single GPU worker thread	~46 photos/min, ~2,700/hr	N workers, one per GPU; autoscale on queue depth	Deliberate today (one GPU, 1.6 GB per BiRefNet)
5	Process-local job table	Dies on restart; breaks with 2 replicas	Redis + RQ. Swap surface is two functions: <code>submit()</code> , <code>get()</code>	Settled, unbuilt
6	No rate limit on <code>/catalog/interpret</code>	Spends money per call	Token buckets + spend ceiling + circuit breaker	Known gap
7	GET	Demo scaffolding; embeds a	Delete before production	Documented as must-

#	Limit	Threshold	Fix	Status
	/api/enhanced/{path}	DHCP host into DB rows		delete

The honest headline: items 1 and 2 are a **project region setting, not code**. The single largest performance fact in this system is that **compute was 15% of the wall clock** and the remaining 85% was geography. That is a good problem to have — it means the expensive part (model inference at 402 ms) is already fast enough, and the rest is configuration.

At scale, the shape does not change:



Stateless API replicas behind a load balancer; the only stateful things are Postgres, Redis and object storage, all of which are managed products. **We do not need a novel architecture — we need the boring one, correctly placed in Mumbai.**

Q6.2 — "Demo day. A minister's office pushes a notification and forty thousand artisans open the app in ten minutes. What actually happens?"

Think. Answer the arithmetic, then say what the user *sees*, because that is the part that decides whether they ever come back.

Today, with one GPU and one worker thread: enhancement queues at ~46/min. Forty thousand photographs is roughly **fourteen hours** of queue. The API itself would survive far longer than the queue — auth, catalog reads and publish are cheap; **it is the GPU that falls over.**

And what the artisan sees today is the real failure: a poll that stays *queued*, with no position and no estimate. Rule 3 in `CLAUDE.md` says every failure must speak; **a queue that is merely slow is not currently treated as a failure, so it says nothing.** That is the bug.

What we would do, in order of effect per rupee:

1. **Backpressure that speaks.** Above a queue-depth threshold: *"abhi bhī hai — aapki photo taiyar hoke aa jayegi, aap aage badhiye."* **Then let them continue.** Enhancement is asynchronous by contract, and rule 3 already says losing the enhancement must never cost the listing — so the artisan finishes the catalog, prices it, and publishes with their own photo, which the enhanced version replaces when it lands. **This costs**

nothing and removes the failure entirely.

2. **Switch the fleet to BiRefNet-lite.** 273 ms vs 645 ms and 851 MiB vs 1615 MiB — **~2.4× throughput and two workers per card instead of one.** Already benchmarked, already "held in reserve." That is roughly a **5× effective capacity change** for a config flag.
3. **Autoscale GPU workers on queue depth**, on preemptible instances — the queue tolerates preemption because the contract is already asynchronous.
4. **Client-side downscale before upload.** The server reduces to 2000 px anyway; uploading 12 MP is bandwidth the artisan paid for and we discard.

Capacity after that, arithmetically: 1.3 s/photo $\Rightarrow$ one GPU at 50% utilisation $\approx$ **1.3 million photographs/month.** At ten photographs per artisan per month that is **~130,000 artisans per GPU.** Forty thousand photographs in ten minutes needs $\sim$ 9 lite-configured GPUs to clear in real time, or **one GPU and a queue that speaks.** The second option is free.

Breaks when. The database, not the GPU, once the region is fixed and the queue is elastic — publish fan-out writes several rows per product across `Product`, `ProductImage`, `ChannelStatus` and `InventoryLedger`. That is a connection-pool and index problem, and it is the ordinary kind.

Q6.3 — "Multi-tenancy. Fifty agencies, each with their own artisans. How do you stop agency A seeing agency B?"

Today. Designed, partially built. The admin console is multi-tenant by design — *"each agency sees only its own artisans"* (`docs/Master-Technical-Reference.md §3.2`), `Artisan.cluster_id` carries the association, and the console pages (`/artisans`, `/orders`, `/reports`) are scoped.

The honest state: scoping today is application-level — a filter in a query. **That is one forgotten WHERE clause away from a cross-tenant leak**, and on a government deployment holding beneficiary data that is the single most damaging bug class available to us.

At scale, the correct answer is not more careful queries: Postgres **row-level security** with the tenant id set per connection, so a missing filter fails closed rather than open. Plus per-tenant audit logging, and a test that asserts a cross-tenant read returns zero rows. RLS is a migration and a session variable, not a rewrite.

Q6.4 — "This is beneficiary data for a social justice ministry. What are you storing about a person, and what happens when it leaks?"

Think. This is the question where our design is genuinely unusual, and it should be stated as a principle before a list.

The golden rule (`docs/Master-Technical-Reference.md §14.1`):

What we don't store cannot leak.

What we NEVER store: PAN number · Aadhaar number · GST certificate · bank account number · photographs of any document · platform passwords · platform OAuth tokens (*one exception: Amazon/Flipkart refresh tokens if the artisan explicitly connects — encrypted*).

What we DO store: phone + OTP session · language preference · display name · UPI id (payout only) · **boolean readiness flags** · product catalog · self-reported outcomes.

The boolean design is the key decision and it is worth one full sentence in the pitch: `has_pan: true` — **never the number**. Four booleans and a phone number is the entire signup. **That keeps us out of DPDP's heaviest obligations entirely**, because there is no financial identifier in the database to be obliged about.

Four design principles behind it:

1. **Never ask for credentials.** OAuth redirect only. **A demo that shows "enter your Amazon password" is a disqualification-level flaw** — and we say that in the adapter's own docstring.
2. **Verify, don't store.** If PAN/GST verification is ever needed: a regulated KYC provider, and we keep `verified: true` + timestamp + masked last-4. Never the full number.
3. **[do not] Do not touch Aadhaar.** The Aadhaar Act restricts storage. If ever unavoidable, Offline eKYC XML or Virtual ID only. **Simplest advice: avoid entirely** — and we do.
4. **Bank verification via penny-drop** (₹1 test transfer), token stored, not the account number.

On a breach. DPDP is two-stage: notify the Data Protection Board **immediately**, then affected Data Principals **within 72 hours**, in plain language, saying what was exposed and what to do. Penalties reach **₹250 crore** for serious violations and **₹200 crore** for failure to notify.

What a breach of our database would actually expose: phone numbers, names, pincodes, UPI ids, product catalogs. **Serious, and worth saying so** — a phone number plus a craft plus a pincode is identifying. But no PAN, no Aadhaar, no bank account, no documents. **The blast radius is a deliberate design output, not luck.**

Q6.5 — "DPDP compliance. Give me a date and an obligation, not a paragraph."

Today. DPDP Rules were **notified 14 November 2025**, phased:

- **November 2026 — Rule 4, Consent Management. First hard deadline.**
- **May 2027 — Rules 3, 5–16, 22, 23:** notices, security safeguards, breach notification, data erasure, children's data, rights management, cross-border transfers.

What is built against it:

- **/consent** is screen 2 of onboarding, before authentication. It stores a **consent artifact**: `{ timestamp, language, notice_version }`. Versioned, so a notice change is auditable.
- **/settings** carries **"mera data mitaayein"** — the DPDP erasure right, as a spoken option in the app rather than an email address in a policy nobody can read.

- The booleans-only design above.

The tie-in that almost nobody else will have thought of (docs/Master-Technical-Reference.md §14.6): DPDP requires the consent notice in **plain language**, with the user able to choose from the **22 Eighth Schedule languages**. Our app is already multilingual. So the notice is **played by voice, in Odia or Hindi or Bengali** — "hum aapka PAN sirf GST enrolment ke liye istemaal karenge, kisi aur ke saath share nahi karenge."

One feature, two wins: DPDP compliance AND accessibility. For a user who cannot read, a written consent notice is not consent. Voice consent is not a nice-to-have here — it is arguably the only valid consent this user can give.

What is NOT closed, and we should say it before we are asked:

1. **The cross-border disclosure.** The notice does not currently say a third-party processor may see the artisan's name during onboarding (Part 3, Q3.6). **Under DPDP that disclosure should be made.** It is the gap I would fix first.
2. **No named Data Protection Officer, no grievance mechanism, no retention schedule.** All three are May-2027 obligations. None exists today.
3. **DigiLocker** is the right answer wherever verified documents are genuinely required — consent-based government fetch, no document photographs. **Requester-organisation registration is a real gate** and is unverified (§18 item 6).

Q6.6 — "Nothing here is observable. How would you even know your app is failing for a weaver in Sambalpur?"

Think. Concede — there is no telemetry stack today — but the *design* already specifies the right metrics, which is the more interesting half.

Today. No metrics pipeline, no tracing, no error aggregation. That is a real gap and it is not written down anywhere as one, which makes it worse than the gaps that are.

But the metrics that matter are already defined by design decisions, which is unusual and worth showing:

Metric	Why it exists	Where it comes from
Gate false-reject rate	21/93 today. If it drifts up, artisans are being told to reshoot good photographs	images/check.py + calibrate.py against fixtures
Selector-pack match rate per step	Below threshold → auto-revert to guided-paste and alert. The channel heals itself	Designed in §6.4, rule 4

Metric	Why it exists	Where it comes from
Enhancement queue depth	The honest autoscale signal, and the backpressure trigger	<code>ai/enhance/jobs.py</code>
Cost per listing	Alerts on spend velocity, not request count	LLM + ASR call sites
<code>artisan_confirmed_payment</code>	The "Paisa aaya?" tap — see below	Order schema
Orders per artisan per channel	Tests the entire thesis. If listings rise and orders do not, cataloging was not the bottleneck	Order

The last two are the ones I would put on a slide, because they are impact metrics that a ministry can audit, not vanity metrics.

And one of them turns a limitation into an asset. We can see marketplace settlement *schedules and promises*. We cannot see the artisan's bank account. So `/earnings` adds a single tap — "Paisa aaya?" → yes/no. One tap gives us **real settlement-delay data across thousands of artisans**, per channel, which is genuinely valuable evidence to hand back to the ministry and which nobody currently has.

Turn the limitation into our best dataset.

Q6.7 — "Who does quality control? A buyer receives a badly made piece and blames the platform."

Think. Say we cannot, immediately. Inventing an answer here is worse than admitting the limit — that instruction is in our own docs.

Answer. We cannot. A software team cannot do physical QC at scale, and any claim otherwise collapses on the first follow-up.

The real answer is three layers, and only one of them is ours:

1. **The cluster / Common Facility Centre does physical QC.** They already do it for exhibitions, and it is the natural function of the Cluster Hub — **and these centres already exist as funded government infrastructure** (Block Level Clusters, Handicraft Service Centres). **We plug into what the government already paid for rather than building a QC operation.**
2. **Platform-level:** buyer ratings, returns tracking, delisting on repeat complaints. Ours.
3. **Certification marks** where held — Handloom Mark, India Handmade Mark, Craft Mark, Silk Mark, GI tags.

The Cluster Hub is worth a slide of its own, because one intervention solves six problems at once: quality check by a human · professional packing by one trained person instead of fifty artisans guessing · accurate weight and

dimensions (under-declared weight silently eats margin through courier surcharges) · **one courier stop for fifty artisans instead of fifty rural pickups** · a local job · and it uses existing infrastructure.

■ *Don't build a logistics company. Plug into what exists.*

Q6.8 — "Counterfeits. What stops someone selling a powerloom saree as handloom on your platform?"

Think. Split the question — the mentor version of this conflated two different problems.

(a) **Someone copies the artisan's design** → IP protection. Very hard, arguably out of scope. **We say so honestly** rather than gesturing at it.

(b) **Fake products sold as authentic on our platform** → solvable, **and the government already solved it.** Handloom Mark, India Handmade Mark, Craft Mark, Silk Mark, GI tags — and the **Handlooms (Reservation of Articles for Production) Act, 1985**, which reserves 11 specified textile articles for handloom production.

■ *Do not invent a new authenticity system. On a government PS, "we surface the Handloom Mark and GI status that DC-Handlooms already issues" beats any novel scheme we could design.*

`certifications[]` and `gi_claim` are fields on `Product` from day one.

Q6.9 — "Add blockchain. Make it decentralised and tamper-proof."

Think. This arrives from mentors and judges constantly. The answer must be respectful, correct, and must end with something we will actually build.

The technical truth first. Blockchain cannot verify that a physical saree is handloom. It is a tamper-evident ledger. If someone writes "*authentic Sambalpuri*" about a powerloom fake, **the chain faithfully stores that lie forever.** This is the oracle problem and it is **unsolvable at the chain layer.**

But there is a narrow, honest version worth building — the Digital Product Passport. A signed record containing artisan/Pahchan id, cluster, craft type, materials, GI claim, date and image hashes. **Signed by the cluster office or handicraft board** — the trusted authority that already exists. The hash is anchored to a public chain or transparency log so it cannot be backdated or altered. **A QR code on the physical tag links to it**, and that QR also rides on the shipping label, tying provenance to something the buyer physically holds.

■ *Trust comes from WHO SIGNED, not from the chain. The chain only proves nobody edited it afterward.*

Pitch it as: "*Blockchain-anchored provenance, with certification authority remaining with DC-Handicrafts.*" That satisfies the request, is technically honest, and answers the counterfeit and validation questions in one stroke.

And the scope discipline: [warning] **one slide. Not in the main architecture diagram. Not core.**

`docs/decisions.md` closed it that way deliberately. A blockchain in our core diagram would be the loudest thing on it and the least load-bearing.